VIVA PREPARATION GUIDE

Wealthline

Software Metrics-Driven, AI-Powered Personal Finance Management System

How every calculation works, which numbers (weights and thresholds) sit in the code and why, what every page and button does, and the exact words to use in front of faculty. With real-life (and slightly funny) examples.

Team: K Druthendra 23MIS0213 · G Sai Santhosh 23MIS0227 · P Siva Kiran 23MIS0674 · Shaik Nihal 23MIS0491

Every number in this guide was read from the real source files (file and line given). Demo login: demo@gmail.com / 12345678

How to use this guide

Part 1	The 60-second pitch and a 10-minute demo script (what to click, what to say)
Part 2	**THE CODE: every calculation, weight and threshold** (anomaly detection, forecast, health score, safe-to-spend, auto-budget, subscriptions, goal ETA, what-if, bill scanning, Gemini settings, security)
Part 3	The software-metrics side: GQ(I)M, decision tree, I-MR control chart, correlation, Bayesian network
Part 4	Master table of ALL constants in one page (memorise this one)
Part 5	Every page, every button, with real-life funny examples
Part 6	Questions the faculty may ask, with model answers
Part 7	Honest limitations (say them yourself before they ask: it earns marks)

RULE FOR THE VIVA

Never say "the AI decides". Say: "A simple, explainable formula decides, and AI only writes the English around it." Faculty love formulas they can check. Almost everything in Wealthline is a transparent formula; only bill reading, advice text and Ask AI use Gemini.

WHAT IS "AI" AND WHAT IS "FORMULA" IN OUR APP

Feature	Who does it
Anomaly detection	Our own z-score formula (no AI service)
Next-month forecast	Our own exponential smoothing (no AI service)
Health score, safe-to-spend, auto-budget, subscriptions, goal ETA, what-if	Our own formulas
Reading a bill photo	Google Gemini (vision), then our code validates and checks the budget
Written recommendations and Ask AI	Google Gemini, but only from a summary of the user's own numbers

Part 1: Pitch and demo script

The 60-second pitch (say it almost word for word)

SAY THIS

"Most finance apps only record what you spent; the user has to do all the thinking. Wealthline does the thinking. You photograph a bill; Gemini reads it; *our* code checks the date, detects a duplicate and tells you what saving it will do to that category's budget *before* you save. Our own formulas flag unusual spends using a z-score with a threshold of 2.5, predict next month's spend with exponential smoothing at alpha 0.4, and give a 0 to 100 health score built from four visible weights: 40, 30, 15 and 15. Because this is a Software Metrics course, we also measured our own project: GQ(I)M plan, decision tree for defect-prone modules, I-MR control chart, three correlation methods and a Bayesian network. Everything is deployed live and tested by 48 automated checks."

The 10-minute demo (follow this order)

Min	Click	Say
0:00	Login as demo	"Demo account is seeded with 94 transactions over 6 months."
0:30	Dashboard	Point at 4 coloured tiles, net worth, safe-to-spend (₹603/day), health score 78 and its four bars. "Every point is explainable, here is the formula." (Part 2.3)
2:00	Scan a bill → Supermarket sample	Wait for result. Point at the red line "puts Groceries over its limit". Explain the 3 checks (date, duplicate, budget impact).
4:00	Transactions → filter, search, Export CSV	"Anomalies get a flag: z-score 2.5."
5:00	Budgets → Auto-budget	"Average of last 3 months plus 10%, rounded up to the next 100."
6:00	Goals, Bills & Subscriptions	Goal ETA formula; subscription detector (3+ months, within 10% of median).
7:00	AI Insights → What-if slider	Forecast (alpha 0.4) and the savings simulator.
8:00	Ask AI: "Where did most of my money go?"	"Answers only from a snapshot of the user's own data."
9:00	Metrics Deck, Settings	Control chart week 12 breach; export / clear / delete data.

DEMO SAFETY

Gemini's free quota is shared and a fresh call can take 6 to 25 seconds (it tries up to 4 models). Scan each of the three sample bills once before the viva: results are cached for 30 days, so they then return in about 0.2 s. Ask AI answers are cached for 24 hours per identical question.

Part 2: THE CODE — weights, thresholds and calculations

All server logic is in `app/server`. Pure-formula code is in `lib/ai.js` and `routes/dashboard.js`; Gemini code in `lib/gemini.js`.

2.1 Anomaly detection (the question your madam asked)

File: `app/server/lib/ai.js` lines 15 to 37. **Called from:** `routes/transactions.js` line 41, every time an *expense* is saved (manual entry or confirmed scan).

```
// lib/ai.js
function mean(xs) { return xs.reduce((a, b) => a + b, 0) / (xs.length || 1); }
function std(xs)  { const m = mean(xs); return Math.sqrt(mean(xs.map(x => (x - m) ** 2))); }

function detectAnomaly(newAmount, historyAmounts) {
  if (historyAmounts.length < 4)                    // MIN HISTORY = 4
    return { isAnomaly: false, zScore: 0, reason: "Not enough category history yet to judge." };
  const m = mean(historyAmounts);
  const s = std(historyAmounts) || 1;                 // guard: avoid divide-by-zero
  const z = (newAmount - m) / s;
  const isAnomaly = Math.abs(z) >= 2.5;              // THRESHOLD = 2.5
  ...
}
```

The three numbers that matter

Constant	Value	Meaning	Why this value
Minimum history	`4`	Fewer than 4 past expenses in that category → we refuse to judge	With 1 to 3 points the "average" and "spread" are meaningless (cold-start protection)
Threshold	`|z| ≥ 2.5`	Flag when the new amount is 2.5 standard deviations away from the category average (either direction)	Chosen by simulation (table below): best balance of few false alarms and decent catching
Spread guard	`|| 1`	If all past amounts are identical, std = 0; we use 1 instead of crashing	Division safety

The formula in plain words

z = (new amount − average of that category's past expenses) ÷ standard deviation of those past expenses. Standard deviation is "the normal wobble". z tells you *how many normal wobbles* away from normal this purchase is. Here std is the *population* standard deviation (divide by n, as in line 17).

> **FUNNY EXAMPLE**
> Your chai usually costs ₹20 ± ₹5 (the "wobble"). One day the bill is ₹2,000. Distance from normal = 1,980, and 1,980 ÷ 5 is a z of about 396. Either you bought chai at a five-star hotel, or the chaiwala's QR code is now connected to someone else's phone. Wealthline raises its eyebrow (and a notification) exactly like your mother does when she sees the Swiggy bill.

Worked example with real arithmetic

Groceries history: 900, 1500, 1100, 1800, 1300, 1600. Mean = 1,366.67. Population std = 303.68. Flag line = mean + 2.5 × std = 1,366.67 + 759.2 = **₹2,125.87**.

New grocery bill	z = (x − 1366.67) ÷ 303.68	Flagged?
₹2,000	`2.09`	No (below 2.5)
₹2,600	`4.06`	**Yes**
₹3,500	`7.02`	**Yes**

What happens when flagged: the transaction is saved with `flaggedAnomaly: true` and its `anomalyZScore` (transactions.js line 46); a notification is inserted ("Unusual Groceries transaction of ₹... — This transaction is 7.0x the typical spread for this category (avg ₹1367)"); the Insights page lists the 5 latest flagged items as "critical" recommendations; the Health Score loses 5 points per flagged item this month; and the Ask AI snapshot includes the last 5 flagged items.

The demo case: in the demo account a ₹9,800 grocery bill shows z = 76.42, because the account's normal groceries are very steady, so the "wobble" is tiny.

Why 2.5? (evidence, not opinion). Script: `app/server/scripts/evaluate_models.js`, seed 20261026

We simulated 300 users × 200 transactions, injecting 10% anomalies, and tried three thresholds (results stored in `analysis/eval_models.json`):

Threshold	Precision	Recall	F1	False-alarm rate
2.0	96.1%	72.4%	0.826	0.34%
2.5 (chosen)	**98.4%**	**63.1%**	**0.769**	**0.11%**
3.0	99.4%	53.5%	0.696	0.04%

Precision = of the things we flagged, how many were truly unusual. **Recall** = of all truly unusual things, how many we caught. In a finance app a false alarm annoys users and makes them ignore alerts, so we favour precision (98.4%) over catching everything. Recall by size of the spike at 2.5: spikes of 1.5 to 2 times normal are caught only 6.7%; 2 to 3 times 28.1%; **3 to 5 times 94.9%**. So "we catch big, obviously unusual spends reliably; subtle ones we deliberately leave alone."

> **SAY THIS IF ASKED "WHY NOT 2 OR 3?"**
> "At 2 we catch more but false alarms triple (0.34% vs 0.11%). At 3 we are almost never wrong but we miss nearly half. 2.5 is the knee of that trade-off, and we measured it, we did not guess."

> **KNOWN WEAKNESS (ADMIT IT)**
> A huge past outlier inflates the standard deviation and can hide later ones (called masking). A robust version would use the median and MAD. Also, anomaly is per category and amount only: it does not look at time of day, merchant or location.

2.2 Next-month forecast: exponential smoothing

File: `lib/ai.js` lines 60 to 76. Used by the Insights page, the "forecast over budget" recommendation, the what-if simulator and the Ask AI snapshot.

```
const SMOOTHING_ALPHA = 0.4;
function forecastNextMonth(transactions) {
  const series = monthlyTotals(transactions);            // total spend per month, oldest first
  if (series.length === 0) return { forecast: 0, method: "none", confidence: "low" };
  if (series.length < 3) return { forecast: mean(series), method: "average", confidence: "low" };
  let level = series[0].total;
  for (const { total } of series.slice(1))
    level = 0.4 * total + 0.6 * level;                   // the whole algorithm
  return { forecast: Math.max(0, level), method: "exp-smoothing",
           confidence: series.length >= 6 ? "medium" : "low" };
}
```

Constant	Value	Meaning
alpha	`0.4`	How much weight the **latest month** gets (40%); the history so far gets 60%
Minimum months	`3`	Under 3 months we just use the plain average and mark confidence "low"
Medium-confidence cut	`6 months`	6 or more months of data → "medium"; never "high" because it is only a statistical smoother
Floor	`0`	A forecast can never be negative

Worked example (monthly grocery totals 100, 120, 110, 130, 125, 140; in ₹ hundreds): start level = 100; then 0.4×120+0.6×100 = 108.0; then 0.4×110+0.6×108 = 108.8; then 0.4×130+0.6×108.8 = 117.28; then 0.4×125+0.6×117.28 = 120.37; then 0.4×140+0.6×120.37 = **128.22**. So next month is forecast at about 128 (the latest 140 is pulled back toward the trend, which is what smoothing is for).

FUNNY EXAMPLE

alpha = 0.4 means: "40% believe last month's gossip, 60% believe the old wisdom." If you set alpha = 1 you believe only last month, like a friend who thinks one good IPL match makes a team world champions. If alpha = 0 you never update, like an uncle who still thinks petrol is ₹60.

Why smoothing and not regression? (we tested)

Original code used a linear-regression line. We measured MAPE (average percentage error) on 300 synthetic series × 8 forecasts = 2,400 forecasts:

Method	MAPE
Exponential smoothing (alpha 0.4)	**13.76%**
3-month moving average	14.05%
Linear regression	15.95%
Naive (copy last month)	17.04%

Smoothing beat regression in 60% of series and naive in 63.4%. It also won on rising, flat and falling trends. So we replaced regression with smoothing. **MAPE** = mean of |actual − forecast| ÷ actual, as a percent.

HONEST NOTE

Alpha 0.4 was a reasonable fixed choice, not tuned per user. The 3-month moving average is almost as good (14.05%); smoothing wins mainly on robustness and simplicity. Evaluation data is synthetic, so say "on simulated series".

2.3 Financial Health Score (weights 40 / 30 / 15 / 15)

File: `routes/dashboard.js` lines 18 to 46. The most "weights-heavy" calculation: your madam will like this one.

```
savings = clamp(savingsRate / 20, 0, 1) * 40;                  // full marks at 20%+ savings rate
adherence = (budgets within limit / total budgets) * 30;   // 1.0 if no budgets are set
calm = max(0, 15 - 5 * anomaliesThisMonth);                    // lose 5 per flagged anomaly
goalPts = (average of min(1, saved/target) over goals) * 15;  // 0 if there are no goals
score = round(savings + adherence + calm + goalPts);                // 0..100
grade = score>=80 ? "Excellent" : score>=60 ? "Good" : score>=40 ? "Fair" : "Needs attention";
```

Component	Weight (max pts)	Rule	Full marks when
Savings rate	40	savings rate ÷ 20, capped at 1	You save 20% or more of income this month
Budget adherence	30	share of this month's budgets not exceeded	Every budget is at or below 100%
No unusual spending	15	15 minus 5 per anomaly, floor 0	Zero anomalies this month (3 anomalies = 0 points)
Goal progress	15	mean progress over goals	All goals are fully funded

Score	Grade
80 to 100	Excellent
60 to 79	Good
40 to 59	Fair
0 to 39	Needs attention

Worked example = the demo dashboard (score 78, "Good"). Savings rate 42% → min(1, 42/20)=1 → **40**. 5 budgets this month, 4 within limit (Groceries over) → 4/5 × 30 = **24**. 1 anomaly (the ₹9,800 bill) → 15 − 5 = **10**. Goals about 27% funded on average → 0.27 × 15 = **4**. Total 40+24+10+4 = **78**. This is exactly what the bars on the dashboard show (40/40, 24/30, 10/15, 4/15).

FUNNY EXAMPLE

It is a report card with four subjects and unequal marks. Savings is Maths (40 marks, the big one), Budget discipline is Physics (30), "No weird spending" and "Goals" are the two 15-mark practicals. You can score 78 by topping Maths and failing the practicals, just like in real college.

WHY THESE WEIGHTS?

"Saving is the strongest signal of financial health, so it gets the most weight (40). Staying within budgets is the habit that produces savings (30). Anomaly-free spending and goal progress are supporting signals (15 each). The weights are a documented, transparent design choice; they are not learned, and we show each component in the UI so the score is auditable."

2.4 Safe-to-spend and month-end projection

File: `routes/dashboard.js` lines 106 to 126.

```
daysLeft = daysInMonth - dayOfMonth + 1;                    // includes today
remainingBudget = sum over this month's budgets of max(0, limit - spent);
perDay = floor(remainingBudget / daysLeft);                 // null if no budgets are set
typicalMonthlyExpense = average expense of the last 3 full months (months with 0 are skipped);
projectedMonthExpense = spentSoFar + typical * ((daysLeft - 1) / daysInMonth);
onPace = projected <= typical * 1.1;              // UI chip: "On pace" vs "Above usual"
```

Worked example from the demo: ₹3,622 left across 5 budgets, 6 days left → 3622 ÷ 6 = 603.67 → floor = **₹603 per day**. Projected spend ₹45,509 is more than 1.1 × the typical month, so the chip says "Above usual".

Important detail: remaining budget uses `max(0, limit − spent)` per category, so one overspent category never creates "negative money" that cancels another category's leftover.

> **FUNNY EXAMPLE**
> Hostel mess logic: you have ₹3,600 left and 6 days to payday. ₹600 a day. Spend ₹1,500 on one birthday biryani and the app does not scold you, it just quietly tells you tomorrow's budget is Maggi.

2.4b Budget used %: the colours

`percentUsed = round(spent ÷ limit × 100)` (budgets.js line 21). In the UI: over 100% → red "critical"; over 80% → amber "warn"; otherwise green "good" (DashboardView).

2.5 Auto-budget (suggested limits)

File: `routes/budgets.js` lines 26 to 57.

```
avg = average monthly spend of the last (up to) 3 completed months for that category
        (falls back to "this month so far" if no completed month exists)
suggested = Math.ceil( (avg * 1.10) / 100 ) * 100;     // +10% headroom, rounded UP to nearest 100
```

Example: average 4,130 × 1.10 = 4,543 → divide by 100 = 45.43 → ceil = 46 → **₹4,600**. Constants: window 3 months, headroom 10%, rounding step 100.

> **FUNNY EXAMPLE**
> Mom's "extra roti rule": cook for exactly what the family eats, then add 10% in case a cousin drops in, and round up because nobody makes 4.4 rotis.

2.6 Subscription detector

File: `routes/insights.js` lines 147 to 178. A group of expenses with the same description is called a subscription only if **all** are true:

Rule	Value
Seen in distinct months	**3 or more**
Charges per month (average)	**at most 1.5**
Every amount within the median	**±10% (0.1 × median)**

Yearly cost = median amount × 12. "Tracked" = a bill with a similar name already exists. Descriptions are compared case-insensitively. Example: "Netflix" ₹649 in 4 different months → flagged, yearly ₹7,788.

FUNNY EXAMPLE

The "Netflix ghost": the gym membership you joined on 1 January and visited twice. Same amount, every month, never questioned. The detector is the friend who says "bro, you are still paying for that?"

2.7 Goal ETA and the what-if simulator

Goal ETA (`GoalsView.jsx` lines 21 and 45 to 57):

```
avgSavings = average of (income - expense) over the last 3 full months that had income;
months = ceil( (target - saved) / avgSavings );        // ETA = today + months
if deadline exists: monthsLeft = max(1, round(daysToDeadline / 30.4));
   if (months > monthsLeft) -> "behind", neededPerMonth = ceil(remaining / monthsLeft)
```

What-if simulator (`InsightsView.jsx` line 33): `monthly saving = forecast × (cut% ÷ 100)`, slider 5% to 60% in steps of 5; yearly saving = ×12.

FUNNY EXAMPLE

Goa-trip goal: ₹30,000 target, ₹6,000 saved, you save ₹4,000 a month → 24,000 ÷ 4,000 = 6 months. Slider: "if I cut Swiggy (forecast ₹3,000) by 20%, I save ₹600 a month, ₹7,200 a year": that is one extra Goa night, paid by skipping 12 midnight biryanis.

2.8 Bill scanning pipeline (all the checks)

Files: `routes/scan.js` , `lib/gemini.js` , web `ScanView.jsx` . Eight steps:

1. **Photo** in the browser: downscaled before upload; the server accepts only JPEG, PNG or WebP and rejects over 4,000,000 base64 characters (about 3 MB) with HTTP 413.
2. **Cache check:** key = hash(photo + category names); scan results live **30 days**. Same photo = no new API call.
3. **Gemini vision** returns JSON: merchant, date, total, currency, category (must be one of the user's own categories), up to 8 line items, confidence 0 to 1. Temperature **0.1** (near-deterministic), thinking budget 256, max 1,500 output tokens, timeout 45 s.
4. **Not a bill?** Gemini returns `not_a_receipt` → we answer HTTP 422.
5. **Total check:** must be a positive number, else 422.
6. **Date sanity:** if the date format is wrong we use today; if the bill is **more than 400 days old or more than 7 days in the future** we show "Check the date" (catches two-digit-year mistakes such as 18-10-26).
7. **Budget impact:** spent-before + bill total = spent-after; percent before and after; "over by" amount (all computed by our code, not by AI).
8. **Duplicate check:** an existing expense on the **same date** with amount within **₹0.01** → "possible duplicate". The user always confirms before anything is saved, and the confirmed save goes through the normal transaction route, so anomaly detection runs on it too.

Measured accuracy (`scripts/evaluate_scan.js` , `analysis/eval_scan.json`): 10 labelled bills plus 1 non-bill. Total exact: **100%**; category: **100%**; date: **90%**; merchant: **90%**; all four correct: **90%**; the one wrong date was caught by the date check (1/1); non-bill rejected (1/1). Mean latency 157 ms (cached repeat; a first call takes several seconds).

FUNNY EXAMPLE
Gemini is a very fast intern who reads handwriting. We are the manager who checks: "date says year 1926? Sir, were you shopping in the British era?" (date check), "you already paid this biryani yesterday" (duplicate check), "this will blow your Dining budget" (impact). The intern never signs the cheque; you do.

2.9 Gemini settings (every parameter and why)

File: `lib/gemini.js`. The only AI service we call is Google's Gemini REST `generateContent` endpoint. There is no OpenAI anywhere. Key: `GEMINI_API_KEY` in the server environment (never in the repo).

Setting	Value	Reason
Model fallback order	gemini-3.6-flash → 3.5-flash → 3.1-flash-lite → 2.5-flash	Free quota is per model; if one says 429 we try the next. Override with `GEMINI_MODELS`
Retry	1 retry after 1 s on a 5xx overload	Transient "high demand" errors
Per-attempt timeout	25 s (overall budget 40 s default)	UI never hangs
Temperature	scan 0.1, Ask AI 0.3, recommendations 0.4	Reading numbers must be exact; advice text can vary slightly
Thinking budget	128 (recs), 256 (scan, Ask)	Small reasoning budget keeps answers fast and cheap (skipped for "lite" models)
Max output tokens	1,024 recs, 1,500 scan, 900 Ask	Cost control
Cache TTL	scan 30 days; Ask AI and recommendations 1 day	Same data + same question = same answer, zero quota
Failure behaviour	Never throws; returns `reason` (quota, timeout, no-api-key...)	The rule-based parts of the app keep working

Privacy of Ask AI (`lib/snapshot.js`): only a compact summary goes to Gemini: net worth, accounts, this-month totals by category, monthly history, budgets, goals, bills, the last 5 flagged anomalies, forecasts, and the **15 most recent transactions**; the question is limited to 500 characters. The prompt says: answer using ONLY this data, quote exact rupee figures, under 120 words, and say so if the data cannot answer. Recommendations: asked to write 2 to 4 one-sentence, number-specific tips, returned as a JSON array.

FUNNY EXAMPLE
Model fallback is like booking an auto: first driver says "no, too far" (429), you call the second, then the third. Caching is "do not ask the professor the same doubt twice; look at your notes."

2.10 Security settings

Item	Value
Password hashing	bcrypt (bcryptjs), cost factor `10` (`routes/auth.js` line 31)
Session token	JWT, expires in `7 days` (`middleware/auth.js` line 6); every route except sign-in and sign-up needs it

Minimum password	8 characters
Data isolation	every query filters by `userId` from the token
Amount validation	must be a positive finite number (negative amounts return 400); found and fixed by our smoke test
Bill photos	processed in memory; never stored
Secrets	environment variables only (`.env` is git-ignored)
Account deletion	requires password; wipes every collection for that user

2.11 Testing numbers

`app/server/scripts/smoke_test.js` runs 48 checks (demo read-only endpoints, Ask AI, then a throwaway account: signup, duplicate signup, wrong login, accounts, transactions, balance update, negative amount rejection, budgets, goals, bills, scan validation, profile, password, export, clear data, delete account) and passes **48/48** on the local server and on the live Vercel site.

Part 3: The software-metrics side (Modules 3 and 4)

BE CLEAR ABOUT THIS IN THE VIVA

The module and sprint metrics (`data/module_metrics.csv` , `sprint_metrics.csv`) are **synthetic data created for the course**: 12 modules and 16 weekly sprint observations. The *methods* are real; the numbers are illustrative. Say so first; it is worth more than hiding it.

3.1 GQ(I)M: Goal, Question, (Indicator), Metric

A top-down plan: start from a goal (e.g. "make financial advice trustworthy"), ask questions ("how often do users accept recommendations? how wrong is the forecast?"), and attach a metric to each. The app really logs these: accept/dismiss clicks are stored as `recommendation-event` notifications (`insights.js` line 135), which feeds the **Recommendation Acceptance Rate**; forecast error is **MAPE** (measured above); transaction failure rate is the control-chart metric. **Why GQ(I)M:** otherwise teams collect numbers nobody needs.

3.2 Decision tree for defect-prone modules (`analysis/analyze.py` line 152)

Setting	Value
Algorithm	scikit-learn DecisionTreeClassifier
max_depth	**3**
min_samples_leaf	**2**
random_state	**42**
Features	cyclomatic complexity, coupling (CBO), cohesion (LCOM), test coverage %
Label	"High" if defect density per KLOC is above the median (5.28), else "Low"
Training accuracy	**91.7%**
Leave-one-out cross-validation accuracy	**58.3% (majority-class baseline 50%)**
Feature importance	LCOM 0.918, CBO 0.082, complexity 0, coverage 0

Say honestly: depth 3 and min leaf 2 keep the tree small enough to read and stop it memorising 12 points. 91.7% is training accuracy and flatters the model; the fair number is leave-one-out 58.3%, only a little better than coin-flip 50%, because 12 modules is tiny. The point of the tree is *explanation* (low cohesion is the strongest split here), not prediction at scale.

FUNNY EXAMPLE

A decision tree is the 20-questions game: "Is cohesion bad? Yes. Is coupling high? Yes. Then this module is probably the one that will ruin your Friday release."

3.3 I-MR control chart (`analysis/analyze.py` lines 218 to 227)

```
x = weekly transaction failure rate (%)          16 weeks
MR = |x[i] - x[i-1]|                               moving range (week-to-week change)
CL     = mean(x)              = 3.517
UCL_I  = CL + 2.66 * mean(MR)  = 6.556
LCL_I  = max(CL - 2.66 * mean(MR), 0) = 0.477
UCL_MR = 3.267 * mean(MR)
```

Where 2.66 and 3.267 come from: standard control-chart constants for subgroup size 2. 2.66 = 3 ÷ 1.128 (1.128 is the constant d2 that turns an average moving range into an estimate of sigma, so the limits are 3-sigma); 3.267 is the constant D4 for the moving-range chart. Result: **week 12 (7.48%) breaches the UCL of 6.556**: the simulated release regression. Weeks 11 and 13 (5.76, 5.71) are high but inside the limit.

FUNNY EXAMPLE

Hostel mess quality: some days the dal is thin, some days thick, that is normal variation. Then the cook changes and every day is a disaster. The control chart is the mess committee that can tell "normal Tuesday dal" from "the cook was replaced".

3.4 Box plot, correlation, t-test

- **Box plot outliers:** Tukey fences at **1.5 × IQR** beyond Q1 and Q3. Defect density fence is 0.2 to 10.59, delay is −2.58 to 15.03: no module is an outlier.
- **Correlation, three kinds on purpose.** Pearson (straight-line strength), Spearman (rank order, robust to outliers), Kendall (agreement of pairs, best for small samples):

Pair	Pearson r	Spearman rho	Kendall tau
Complexity vs defect density	0.559 (p 0.059)	0.592 (p 0.043)	0.473 (p 0.033)
Test coverage vs defect density	−0.370 (p 0.237)	−0.231 (p 0.471)	−0.152 (p 0.545)
Coupling (CBO) vs delay	0.647 (p 0.023)	0.558 (p 0.059)	0.369 (p 0.098)

Significance level used: $p < 0.05$. Complexity vs defects: Spearman and Kendall are significant, Pearson is borderline, all point the same direction. Coverage vs defects: negative as expected but **not significant**. A Welch t-test of low-coverage vs high-coverage modules (mean defect density 6.31 vs 4.50) gives $t = 1.131$, $p = 0.285$, not significant. **Correlation is not causation.**

FUNNY EXAMPLE

Ice-cream sales and drowning both rise in summer; that does not mean ice cream drowns people. Likewise, complex code correlates with bugs, but we did not prove complexity *causes* them.

3.5 Bayesian network (`analysis/bayesian_network.py`)

A small cause-and-effect graph: Requirement Volatility (R) and Team Experience (T) influence Schedule Delay (S) and Defect Density (D), which influence Project Risk (P). Priors: P(R=High)=0.45, P(T=High)=0.60. Hand-specified conditional probability tables; exact inference by enumeration (Bayes theorem). Results for P(Project Risk = High): baseline **28.1%**; worst case (volatile requirements + inexperienced team) **55.0%**; best case **12.8%**; if delay is already observed High **50.0%**; if defect density is observed High **49.1%**. The same tables are re-implemented in the Metrics Deck so the risk calculator is live.

FUNNY EXAMPLE

You hear "the exam paper is leaked" (evidence). Your belief you will pass changes. A Bayesian network is that updating, but done with numbers. Worst case in our network is "changing requirements + freshers", which is every group project in the last week.

HONEST NOTE

The conditional probability tables are **hand-specified** from the module data distribution, not learned from a large dataset. Say "illustrative, expert-elicited".

Part 4: Master table of every constant

What	Value	File and line
Anomaly min history	4 expenses	lib/ai.js:23
Anomaly threshold	|z| ≥ 2.5	lib/ai.js:29
Std guard	|| 1	lib/ai.js:27
Smoothing alpha	0.4	lib/ai.js:60
Forecast fallback / medium confidence	<3 months mean / ≥6 months medium	lib/ai.js:65, 73
Health weights	40 + 30 + 15 + 15 = 100	routes/dashboard.js:27-34
Savings full-marks rate	20%	dashboard.js:27
Anomaly penalty	5 points each	dashboard.js:30
Grade cut-offs	80 / 60 / 40	dashboard.js:36
Typical monthly expense window	last 3 full months	dashboard.js:114
"On pace" tolerance	projected ≤ 1.1 × typical	DashboardView.jsx:56
Budget colour bands	>100% red, >80% amber	DashboardView.jsx:148-152
Auto-budget window / headroom / rounding	3 months / +10% / up to 100	routes/budgets.js:42, 51
Low-savings recommendation trigger	savings rate < 10%	routes/insights.js:59
Subscription rule	≥3 months, ≤1.5/month, ±10% of median	routes/insights.js:165-167
Anomalies shown in Insights	latest 5	routes/insights.js:43
Goal ETA window / month length	3 months / 30.4 days	GoalsView.jsx:21, 53
What-if slider	5% to 60%, step 5	InsightsView.jsx:86
Scan date warning	>400 days old or >7 days ahead	routes/scan.js:56
Duplicate rule	same date, amount within ₹0.01	routes/scan.js:79
Scan image limit	4,000,000 base64 chars (~3 MB)	routes/scan.js:20
Scan items kept	8 line items	routes/scan.js:88
Scan cache	30 days	routes/scan.js:33
Ask AI cache / question length	1 day / 500 chars	routes/insights.js:183, 186
Ask AI snapshot	15 latest transactions, 5 anomalies	lib/snapshot.js:50-51
Gemini timeouts	25 s per attempt, 40 s total	lib/gemini.js:19, 42
Gemini temperatures	0.1 scan, 0.3 ask, 0.4 recs	lib/gemini.js:103,124,139
bcrypt cost / JWT lifetime / min password	10 / 7 days / 8 chars	auth.js:31, middleware/auth.js:6, settings.js
Decision tree	depth 3, min leaf 2, seed 42	analysis/analyze.py:152

Control chart constants	2.66 (I), 3.267 (MR)	analysis/analyze.py:225-227
Outlier fence	1.5 × IQR	analysis/analyze.py (iqr_outliers)

ONE-LINE MEMORY TRICK

"**2.5, 4, 0.4, 3**": threshold 2.5, minimum history 4, alpha 0.4, minimum months 3. "**40-30-15-15**": health weights. "**3 and 10%**": auto-budget uses 3 months and adds 10%.

Part 5: Every page and every button

Layout: a **left sidebar** (pinned, never covers the page; "Collapse menu" shrinks it to icons and the page slides over), a **top bar** (greeting, assistant toggle, camera, notifications bell, dark-mode moon), the **page**, and on wide screens the **AI assistant panel** docked on the right.

Sidebar and top bar

Control	What it does	Funny real-life version
Scan a bill (purple)	Opens the bill scanner from anywhere	The "Shazam" for receipts
Dashboard, Transactions, Accounts, Budgets, Goals, Bills & Subscriptions	Money pages	Different rooms of one house
AI Insights, Ask AI	Smart tools	The clever relative who knows your account
Metrics Deck	The software-metrics dashboard of our own project	The class topper's marksheet of the app itself
Settings	Profile, password, theme, data tools	Control panel
Collapse / Expand menu	Sidebar to icons and back (remembered)	Folding the dining table to make room
Log out	Ends the session	Locking the cupboard
Sparkle icon (top)	Show or hide the AI assistant panel	Calling the clever relative
Camera icon (top)	Jumps to scanning	Quick-draw for bills
Bell with red dot	Notifications (red count = unread)	The nosy neighbour's alerts
Moon	Light and dark theme	Night mode for 2 a.m. expense guilt

Dashboard

Element	Meaning and source	Funny example
Four pastel tiles	Income, Spent, Savings rate, Safe to spend per day; numbers count up when the page loads. Savings rate = (income − expense) ÷ income	Mint = salary, peach = "where did it go", lavender = how responsible you were, sky = what you may spend today
Net worth card (dark)	Sum of all account balances	Your "if I sold everything" number
Safe to spend today	Formula 2.4; chips: budgets over limit, On pace / Above usual	Hostel-mess daily limit
Income vs expenses chart	Last 6 calendar months, green income, indigo expense	Salary day mountain, rent-day valley
Financial health ring	Score 0 to 100 with four bars (2.3); the chip shows the grade	Report card
Where it went	Donut of this month's expenses by category	The pie your wallet got eaten by

Recent activity / View all	Last 6 transactions; View all opens Transactions	Last night's receipts
Coming up / Bills	Next 4 recurring bills with "Due today / tomorrow / In n days"	Mom's reminder, but automatic
Budgets / Manage	First 4 budgets with colour chips; Manage opens Budgets	Traffic lights for your spending
Scan bill, Add transaction (header)	Open scanner or the add-transaction form	Lazy way and honest way

Scan a bill

Control	What it does
Drop area / Choose file	Pick or drag a JPG, PNG or WebP photo
Take photo	Desktop: live webcam preview with a Capture button. Phone: opens the phone camera. If camera is blocked or missing: friendly message and an Upload fallback
Sample chips (Supermarket, Electricity, Restaurant)	Built-in test bills for demos and for people with no bill handy
Result form	Merchant, total, date, category, pay-from account; all editable. Shows a confidence chip, line items, red/green budget-impact message, "Check the date" and "possible duplicate" warnings
Save expense / Discard / Scan a different bill	Save creates a normal expense (and updates the account balance and runs anomaly detection); Discard throws it away

FUNNY EXAMPLE

Restaurant bill: your friend says "I'll pay, you send me later"; you scan the bill and the app instantly says "this uses 94% of your Dining budget". Friendship preserved, budget scolded.

Transactions

Control	What it does
Search box	Filters by description, category, date or amount
All / Expenses / Income	Type filter
Export CSV	Downloads the visible rows to a spreadsheet
Add transaction	Form: type, amount (positive only), category, account, date, note. Saving expense runs the anomaly check, updates the account balance and may create a notification
Delete (per row)	Removes it and reverses the balance change

FUNNY EXAMPLE

Export CSV is for the day your parents say "show me where the money went" and you want an Excel sheet instead of a speech.

Accounts

Cards for each account (Checking, Savings, Credit Card). **Add account**: name, type, opening balance. Balances move automatically when you add or delete transactions. *Example:* "SBI Credit Card" at ₹0: the account you promised yourself you would not touch.

Budgets

Control	What it does
Set budget	Choose a category and a monthly limit; setting again for the same category updates it
Auto-budget	Opens suggestions (formula 2.5) with checkboxes; **Apply selected** sets them all
Progress bars	Spent / limit; turn red above 100%
Delete	Removes that month's budget

> **FUNNY EXAMPLE**
> Dining budget ₹2,000. Mid-month the bar goes red like your face when someone says "let's just order pizza, it's only ₹300 each".

Goals

Add goal (name, target, optional deadline); **+ Contribute** adds money to a goal; each card shows percent, a progress bar and the ETA line from 2.7 ("On track for Mar 2027" or "needs ₹X a month to meet the deadline"); **Delete** removes. *Example:* "Laptop" goal; the ETA says March 2027, you buy a ₹4,000 game; the ETA slides to May. Instant regret, now in calendar form.

Bills & Subscriptions

Control	What it does
Add recurring bill	Name, amount, day of month, category; appears in "Coming up" on the Dashboard
Delete	Removes a bill
Detected subscriptions table	From formula 2.6, with monthly price, months seen, last charged, yearly cost
Track as bill	Turns a detected subscription into a tracked recurring bill

AI Insights

Element	What it does
Next-month forecast	One row per category with the forecast (2.2), method chip (exp-smoothing or average), confidence
Recommendations	Rules first (forecast over budget, anomalies, savings under 10%), then Gemini lines labelled with the model name
Accept / Dismiss	Logs your choice; this is the Recommendation Acceptance Rate metric from our GQ(I)M plan
What-if savings simulator	Choose a category and a cut percentage; shows monthly and yearly savings (2.7)
Subscriptions card	Same detector output

FUNNY EXAMPLE

Accept = "yes sir, I will stop ordering at midnight" (and then not do it). The app dutifully records the promise; the metric measures how many promises we collect, not how many are kept.

Ask AI (and the docked assistant)

Suggestion chips ("Where did most of my money go this month?", "Am I on track with my budgets?", "How can I save ₹5,000 more every month?", "Which of my expenses is the biggest outlier?", "How long until I reach my savings goals?") or type your own; the animated orb thinks while Gemini answers. Answers come only from your own snapshot (2.9). *Example question:* "Can I afford a ₹30,000 phone?" The assistant looks at your savings, not your emotions.

Notifications

List of anomaly alerts and system notices. **Mark read** clears the red dot. Accept/dismiss clicks are stored too but hidden from the count.

Metrics Deck

The self-contained Module 3 and 4 dashboard embedded in the app (built from `dashboard/template.html` by `build_dashboard.py`): GQ(I)M, decision tree, box plots, I-MR control chart, correlation views, and the live Bayesian risk calculator. This is the page that proves the project is not only an app but also **measured engineering**. *Example:* it is the app's own medical report.

Settings

Control	What it does
Save profile	Change name and email (email already used → error 409)
Update password	Current password required; new one at least 8 characters; confirm must match
Theme: System / Light / Dark	Follows the device or forces a theme
AI assistant: Show / Hide	Controls the docked panel
Download my data	JSON file of everything stored about you
Clear my data	Removes transactions, budgets, goals, bills, notifications, resets balances to 0; keeps login and categories; asks to confirm
Log out	Ends session
Delete my account	Needs your password; permanently removes the account and all data

FUNNY EXAMPLE

"Clear my data" is the "new year, new me" button. "Delete account" is the same, but with a password, because it is the "I am moving to a Himalayan monastery" button.

Part 6: Questions faculty may ask, and answers

Q1. How do you detect anomalies in the budget? Show the threshold in the code.

Open `lib/ai.js` line 29. We compute z = (amount − category mean) ÷ category standard deviation and flag when |z| ≥ 2.5, but only if the category has at least 4 past expenses. 2.5 was chosen from a simulation of 300 users × 200 transactions: precision 98.4%, recall 63.1%, false-alarm rate 0.11%; thresholds 2 and 3 were worse trade-offs.

Q2. What is a z-score and why use it?

The number of standard deviations a value is from the mean. It is unit-free, cheap, explainable to a normal user ("7x the usual spread") and needs no training data, which matters because each user starts with no history.

Q3. Why not machine learning for anomalies (isolation forest, autoencoder)?

A new user has 0 to 20 transactions; ML needs far more. A z-score works from 4 points, is explainable and runs inside a serverless function. ML is listed under future work once enough data exists.

Q4. Is the anomaly check per user or global?

Per user and per category (history is filtered by `userId` and `categoryId`, transactions.js lines 38 to 40). Rent of ₹18,000 is normal for Rent and not compared with Groceries.

Q5. What are the weights in your health score and why?

40 savings rate, 30 budget adherence, 15 no unusual spending, 15 goal progress, total 100 (dashboard.js lines 26 to 46). Savings gets most weight because it is the outcome that matters; budget adherence is the habit that causes it; the other two are supporting signals. They are a transparent design choice and are shown component by component in the UI; they are not learned.

Q6. How is the forecast computed and how accurate is it?

Simple exponential smoothing, alpha 0.4, on monthly category totals; below 3 months it uses the average and says "low" confidence. On simulated data MAPE is 13.76%, better than regression 15.95%, moving average 14.05% and naive 17.04%.

Q7. Why alpha = 0.4?

It balances reacting to a real change (higher alpha) with ignoring one-off noise (lower alpha). We fixed it for transparency and validated the resulting method against alternatives. Tuning alpha per user is future work.

Q8. Which software metrics did you use, and how do they appear in the system?

Product metrics (complexity, CBO, LCOM, coverage, defect density) for the decision tree and correlations; process metrics (weekly transaction failure rate) for the I-MR chart; AI-quality metrics (MAPE of forecast, precision/recall of anomaly detection, recommendation acceptance rate, scan field accuracy). The last group are measured on the real code and logged by the real system; they are the GQ(I)M "metrics" in practice.

Q9. Is your data real?

The finance data in the demo is seeded sample data. The module/sprint metrics are synthetic, generated for the course by `analysis/generate_data.py`. The evaluation of forecast and anomaly methods uses seeded simulation (seed 20261026) so anyone can reproduce it. Scan accuracy uses 10 labelled bill images in `report/eval_receipts`.

Q10. Which AI API do you use? Is the user's data safe?

Google Gemini REST API only (model chain gemini-3.6-flash → 3.5-flash → 3.1-flash-lite → 2.5-flash). Bill photos are processed in memory and not stored. Ask AI receives only a compact summary of that user's own data (15 latest transactions plus aggregates). Key is a server environment variable, never in the repo.

Q11. What if Gemini is down or the quota ends?

The model chain falls back to the next model; results are cached (30 days for scans, 1 day for advice); the functions never throw, they return a reason, and the UI shows a friendly message and the manual add path. The formula-based features (anomaly, forecast, score) do not depend on Gemini at all.

Q12. How do you stop the AI from making up numbers on a bill?

Temperature 0.1, a strict JSON format, and our own checks: the total must be positive, the date is sanity-checked (over 400 days old or over 7 days ahead is flagged), duplicates are detected, and the user must confirm and can edit every field. We measured 100% total accuracy and 90% all-fields on 10 labelled bills.

Q13. Explain the control chart limits.

Centre line is the mean failure rate 3.517%. UCL = CL + 2.66 × average moving range = 6.556%, LCL = 0.477%. 2.66 = 3 ÷ d2 (1.128). A point outside the limits (week 12 at 7.48%) signals a special cause, not normal variation.

Q14. Your decision tree has 91.7% accuracy; is that good?

That is training accuracy and over-optimistic. Leave-one-out accuracy is 58.3% against a 50% baseline, so it is only mildly better than guessing, as expected with 12 modules. Its value is explanation (LCOM importance 0.918), not prediction.

Q15. Why three correlation coefficients?

Pearson assumes linearity and is sensitive to outliers; Spearman uses ranks; Kendall counts concordant pairs and suits tiny samples. When all three agree on direction (as for complexity vs defects) the conclusion is more trustworthy.

Q16. How is the budget affected when a bill is scanned?

The server finds that category's budget for the bill's month, adds the bill total to month-to-date spend, and returns before/after percentage and the overspend amount. The scan alone changes nothing; only Save does.

Q17. How do you secure user accounts?

bcrypt (cost 10) hashed passwords, JWT with 7-day expiry, per-user data filtering on every query, positive-amount validation, password-confirmed account deletion.

Q18. How did you test it?

A 48-check automated smoke test (local and production, all pass), a labelled bill evaluation, a model evaluation script with fixed seed, and responsive-layout audits at six screen widths (1920 to 360 pixels).

Q19. What is the architecture?

React single-page app on Vercel → one serverless Express function (`/api`) → MongoDB Atlas; Gemini called only from the server; Python scripts produce the metrics analysis and the embedded dashboard offline.

Q20. What is novel here?

(1) Scan → budget-impact preview before saving; (2) every score is a visible formula; (3) the AI layer is measured with software-metrics methods (precision/recall, MAPE, acceptance rate, control chart) instead of

just being used; (4) the project's own engineering metrics live inside the product.

Q21. What would you improve?

Use robust statistics (median/MAD) for anomalies; per-user alpha tuning; real production metrics instead of simulated ones; add recurring-bill auto-posting, multi-currency and bank import; learn the health-score weights from user outcomes.

Part 7: Honest limitations (say these first)

- **Synthetic metrics data:** module and sprint metrics, and the forecast/anomaly simulations, are generated. Methods are real; absolute numbers are illustrative.
- **Small-sample decision tree:** 12 modules; LOOCV 58.3% only slightly beats the 50% baseline.
- **Z-score masking:** large past outliers inflate the standard deviation and can hide later anomalies; spikes under 2x normal are mostly not caught (recall 6.7% for 1.5 to 2x).
- **Fixed hyper-parameters:** alpha 0.4, threshold 2.5 and the health weights are fixed by design, not personalised.
- **Gemini dependency:** scanning and advice need a working API key and free-tier quota; first call can take up to 25 seconds. Results are cached and fallbacks exist.
- **Scan accuracy sample:** only 10 labelled bills; date and merchant were 90%, not 100%. Real-world messy photos may do worse.
- **Bayesian network tables:** hand-specified, not learned.
- **Camera:** live webcam path tested only up to the permission prompt in automation; confirm once on a real laptop before the viva.

> **FINAL TIP**
> If you do not know an answer, say: "That value is in `lib/ai.js` line 29; I can open it", then open it. Showing the code beats guessing. Faculty are checking that *you* understand *your* code, and every number here is yours.

Generated from the Wealthline source at E:\Software-Metric. If any constant in the code is changed, update Part 4.